

Algoritmos y Estructuras de Datos

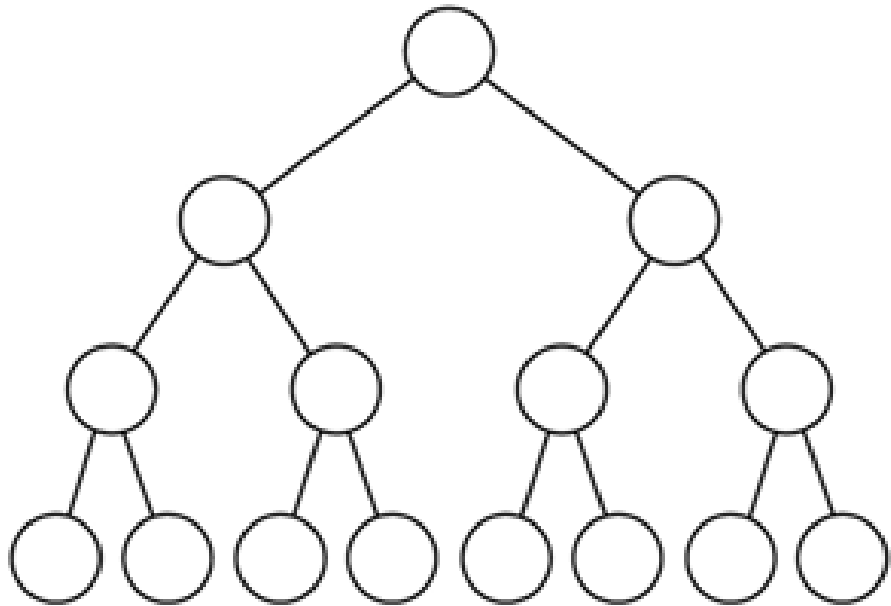
1er semestre - 2026

Árboles balanceados

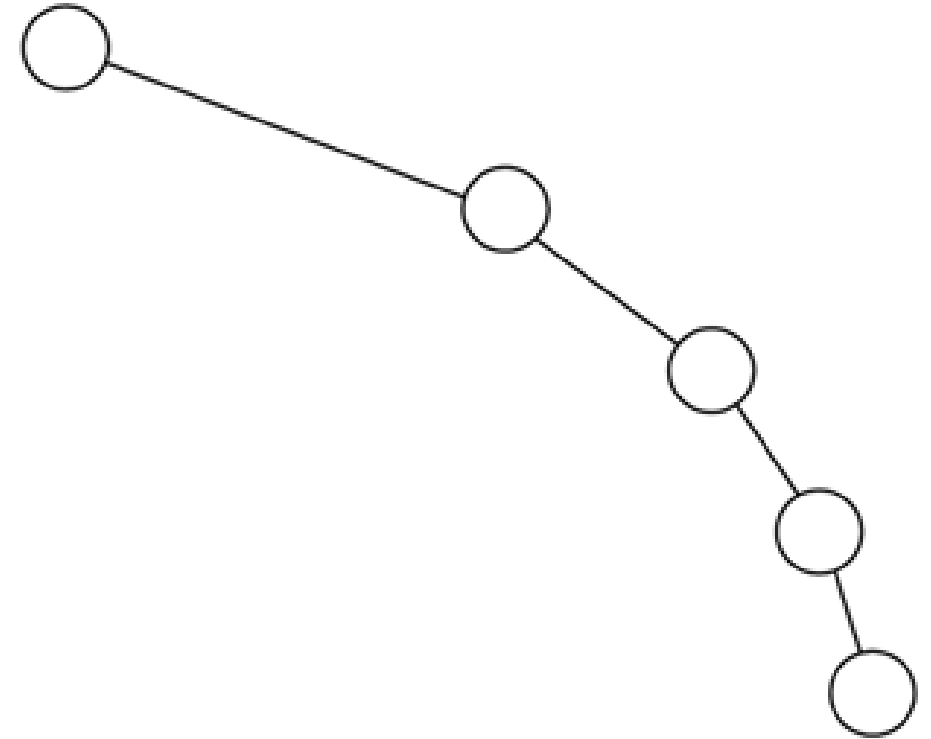
- Criterio Adelson, Velskii, Landis:
 - Un árbol está balanceado si y sólo si para cada nodo las alturas de sus dos subárboles difieren a lo sumo en 1.
 - Asegura un $O \log(n)$ en el **peor caso** para las tres operaciones: **búsqueda, inserción y eliminación**.
 - El teorema de Adelson-Velski y Landis garantiza que un árbol balanceado nunca tendrá una altura mayor que el 45 % respecto a su equivalente perfectamente balanceado.
 - Si la altura de un árbol balanceado de n nodos es $hb(n)$, entonces $\log(n+1) \leq hb(n) \leq 1.4404 \cdot \log(n+2) - 0.328$

Estructuras Jerárquicas vs. Árboles

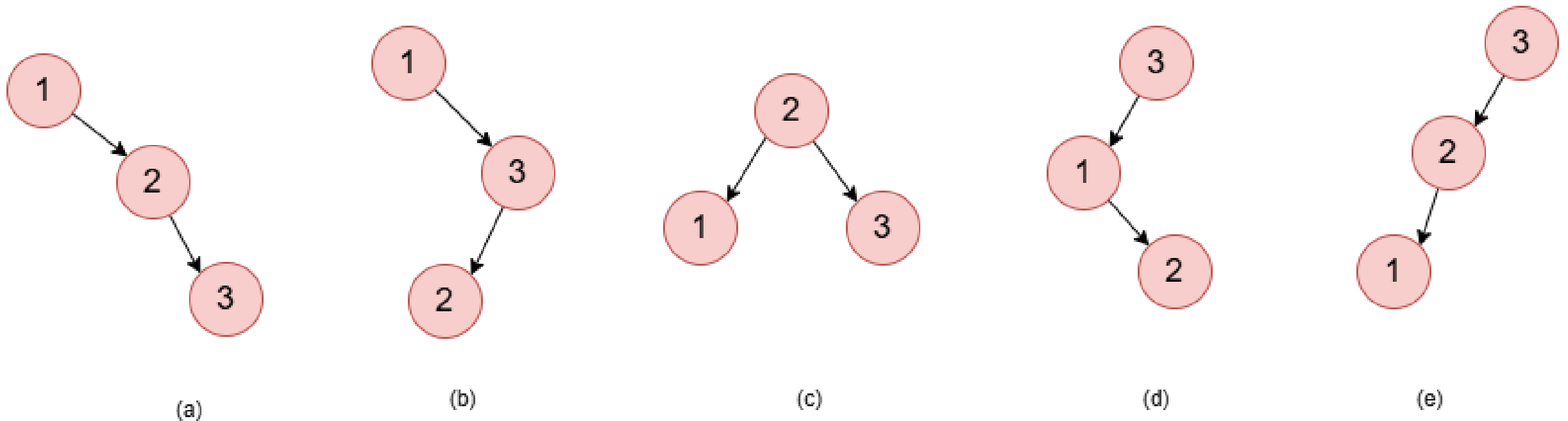
Árbol balanceado altura $\log N$



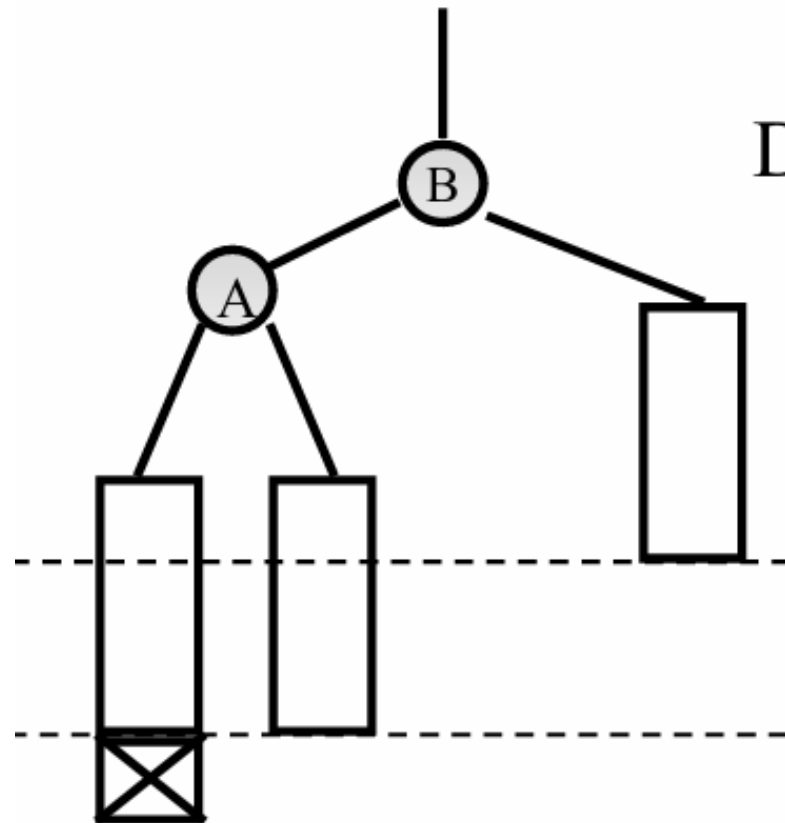
Árbol no balanceado, altura $N - 1$



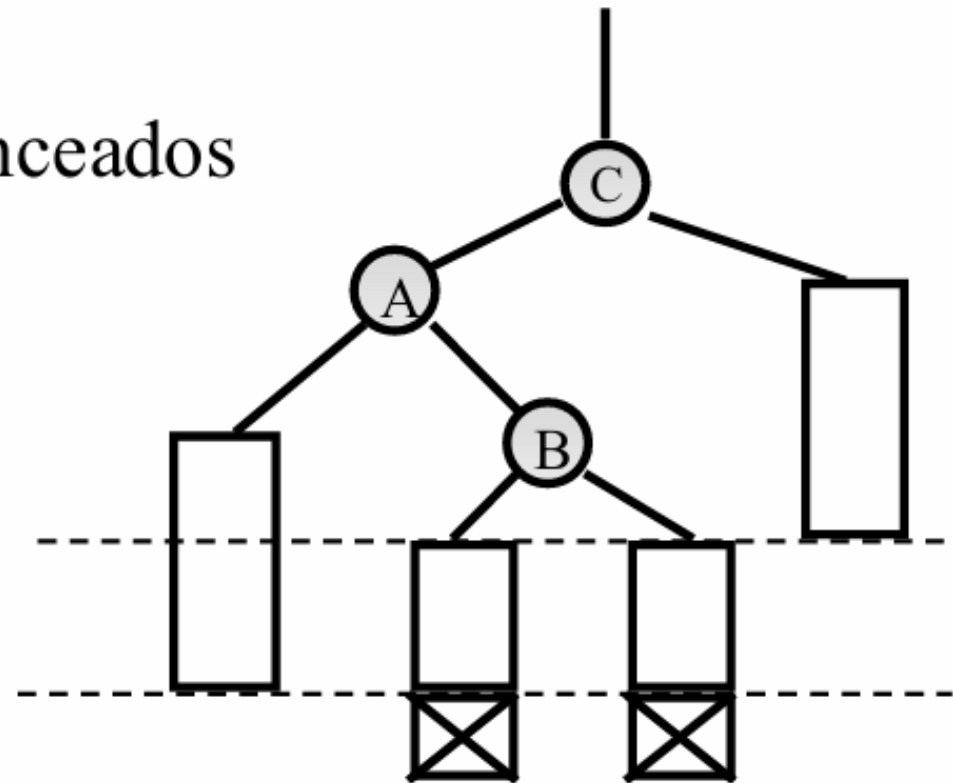
Árboles binarios de búsqueda posibles insertando las claves 1, 2 y 3 de diferentes maneras



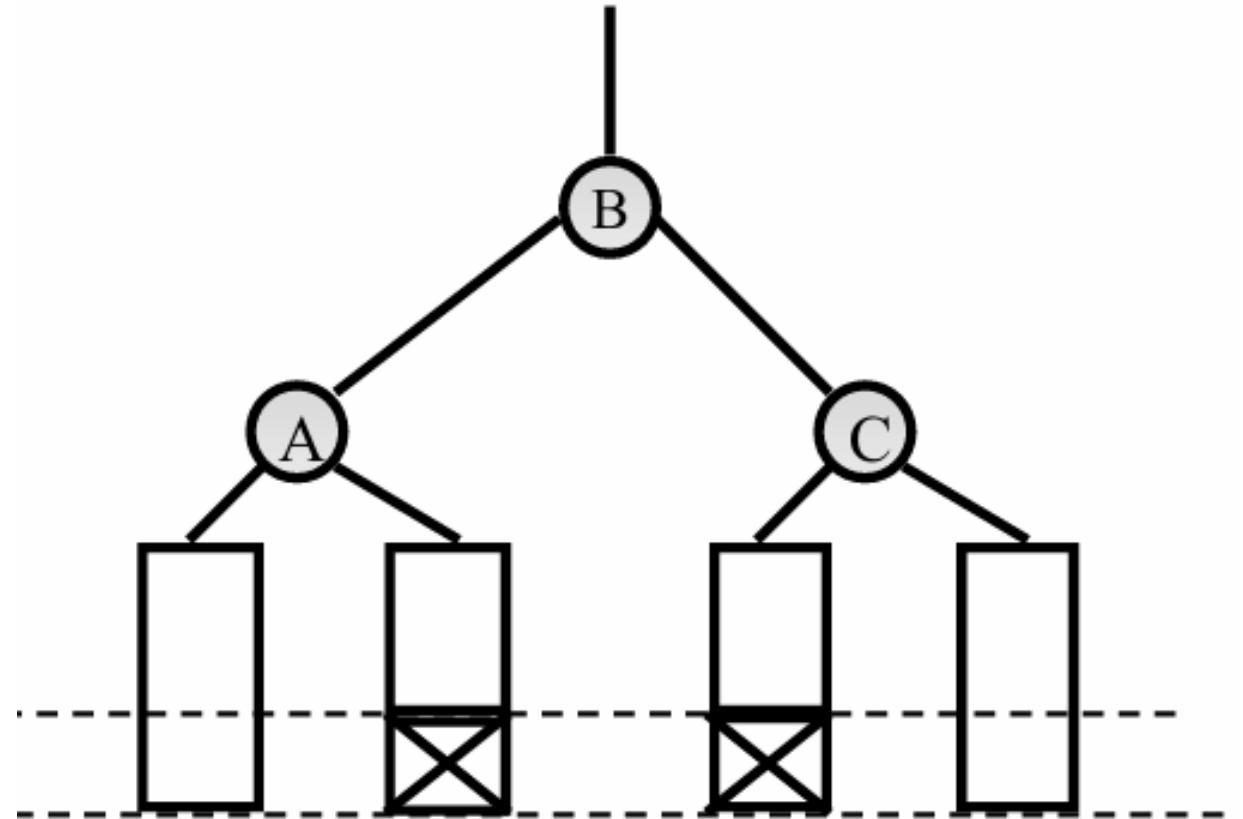
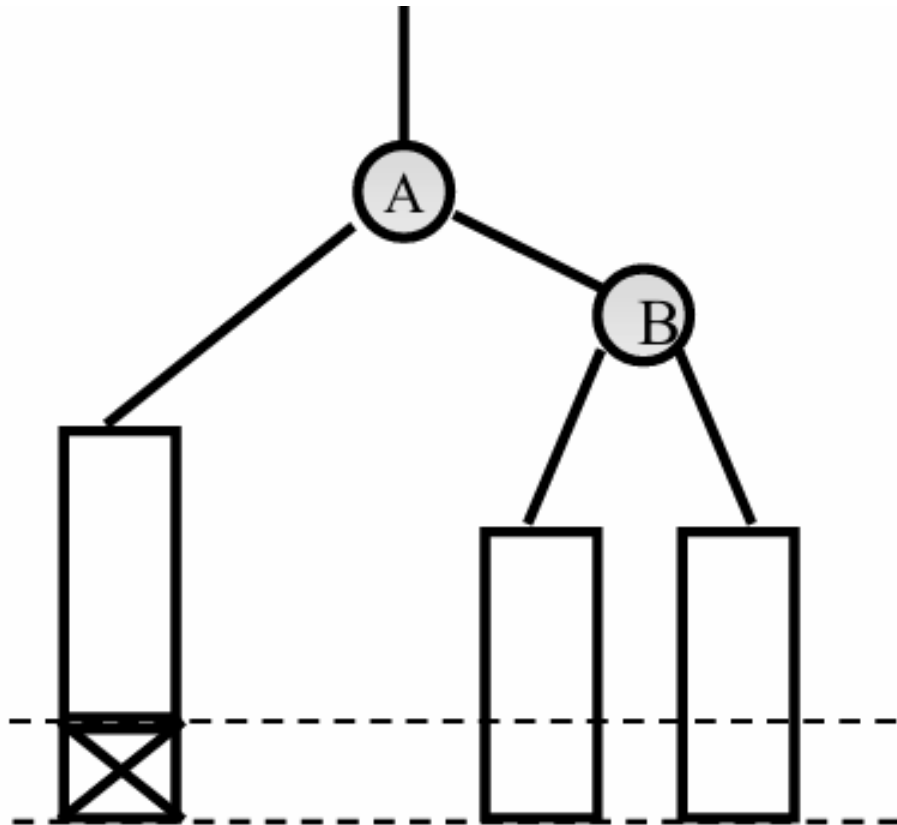
Árboles desbalanceados



Desbalanceados



Árboles balanceados



Árboles balanceados

- Criterio Adelson, Velskii, Landis:
 - Un árbol está balanceado si y sólo si para cada nodo las alturas de sus dos subárboles difieren a lo sumo en 1.
 - Asegura un $O(\log(n))$ en el peor caso para las tres operaciones: búsqueda, inserción y eliminación.
 - Se demuestra con ayuda de los árboles de Fibonacci.

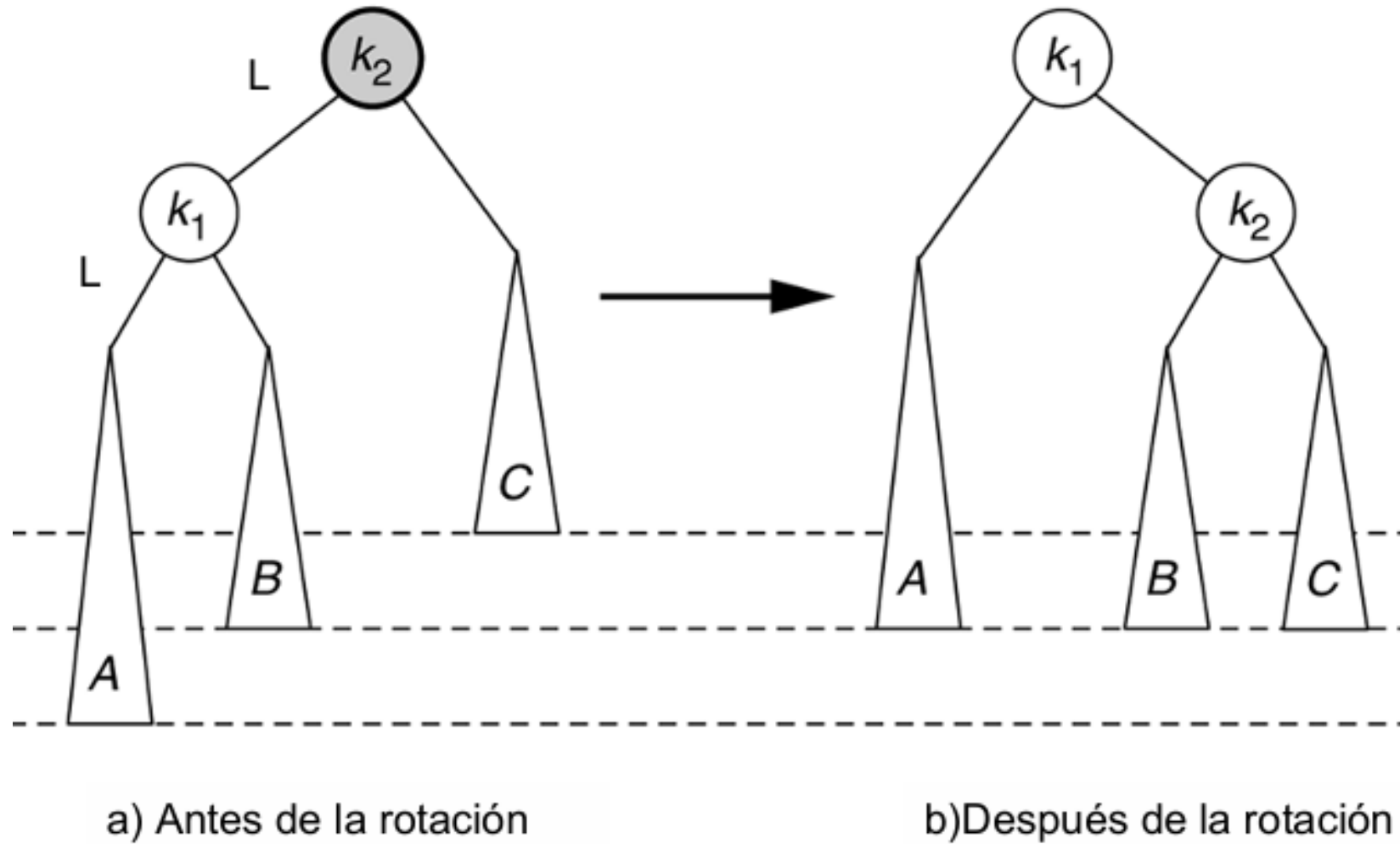
AVL

- Una inserción o eliminación puede destruir el balance
- Se debe revisar la condición de balance y corregir si es necesario
- Después de la Inserción, sólo los nodos que se encuentran en el camino desde el punto de inserción hasta la raíz pueden tener el balance alterado
- Si se repara correctamente el equilibrio del nodo desequilibrado más profundo, se recupera el equilibrio de todo el árbol

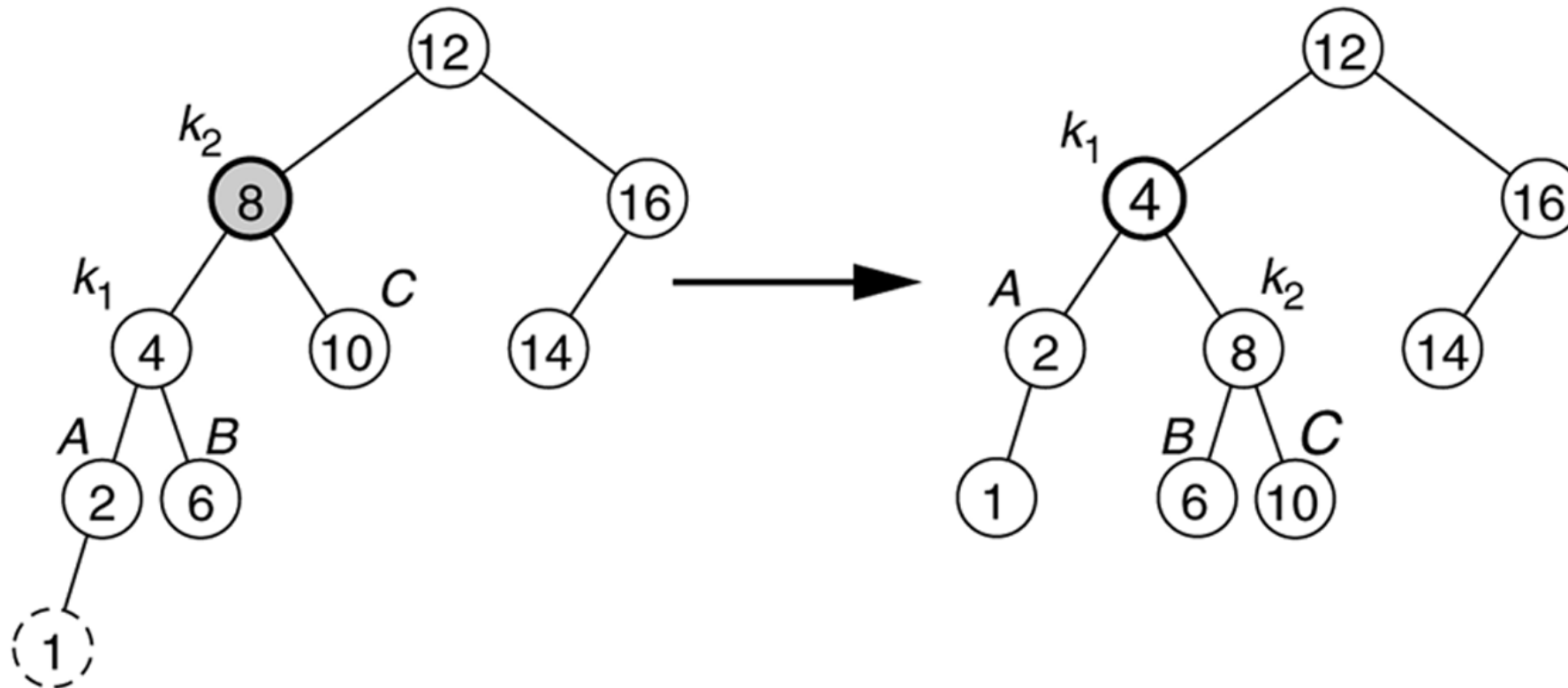
Diferentes condiciones de desequilibrio

- Supongamos que X es el nodo cuyo equilibrio queremos ajustar
- Se pueden dar cuatro casos
 1. Inserción en el subárbol izquierdo del hijo izquierdo de X
 2. Inserción en el subárbol derecho del hijo izquierdo de X
 3. Inserción en el subárbol izquierdo del hijo derecho de X
 4. Inserción en el subárbol derecho del hijo derecho de X
- 1 y 4 son simétricos respecto a X
- 2 y 3 son simétricos respecto a X

Rotación simple para resolver el caso 1



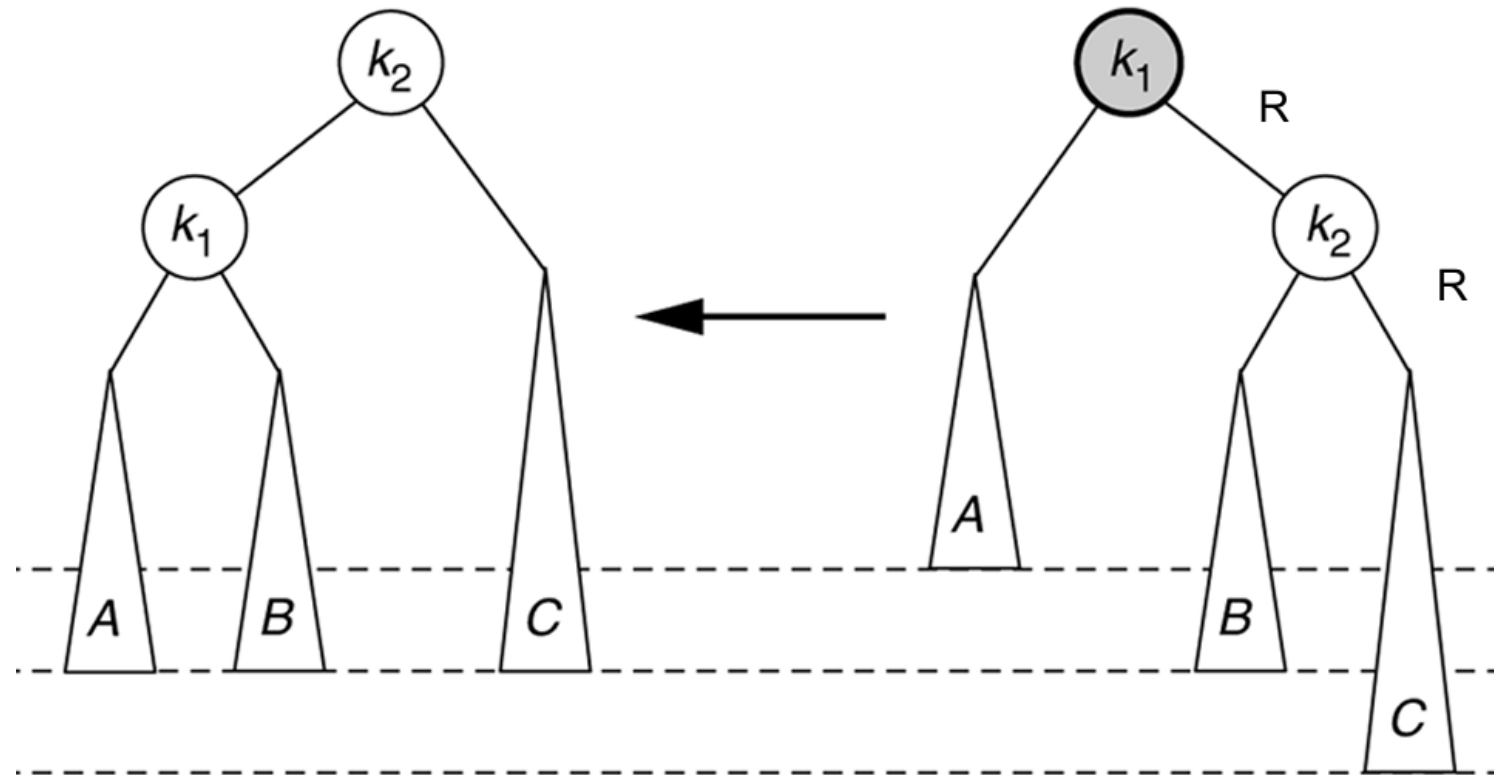
La rotación simple balancea el árbol después de la inserción del 1.



a) Antes de la rotación

b) Después de la rotación

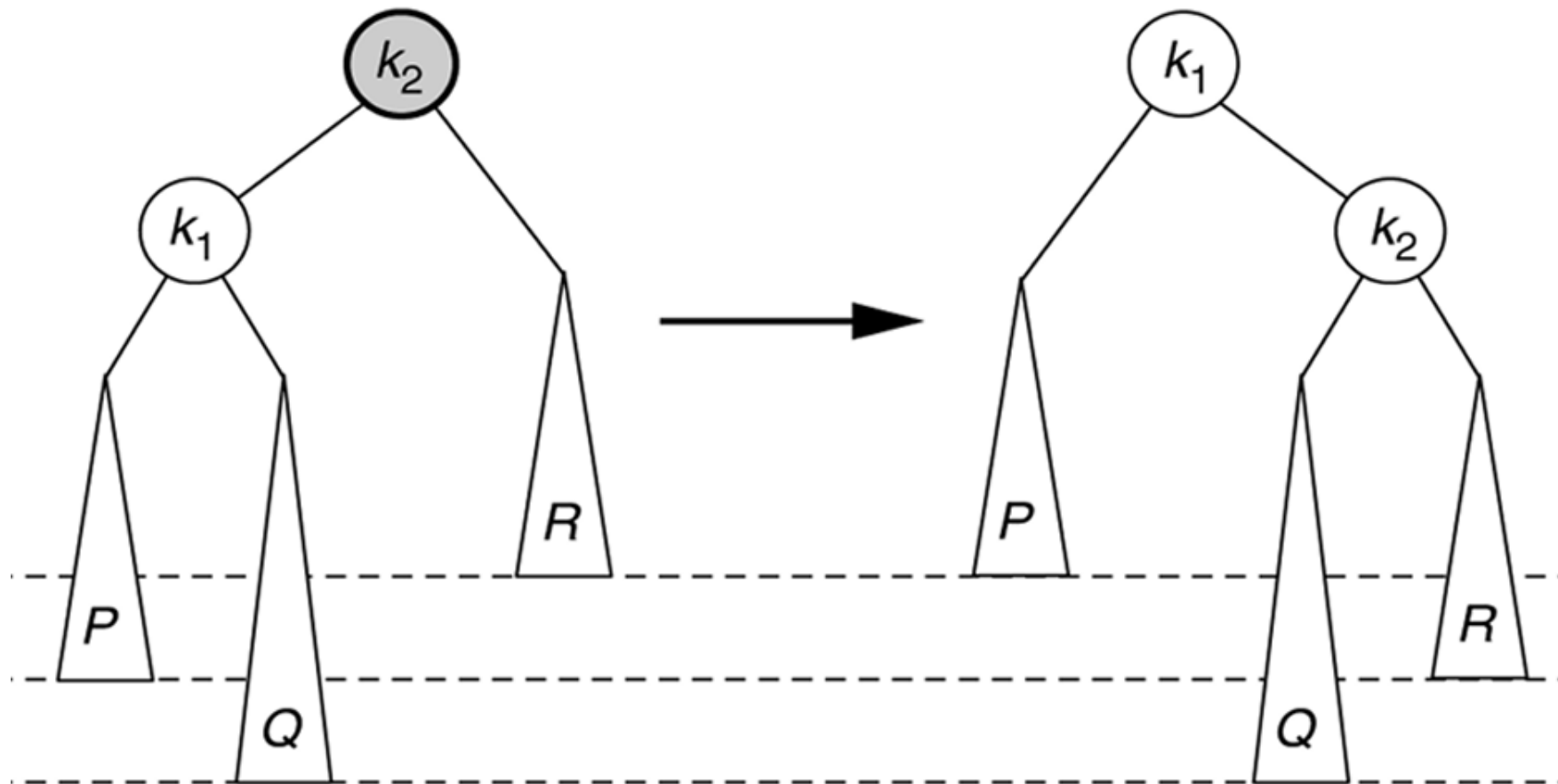
La rotación simple simétrica para resolver el caso 4.



a) Después de la rotación

b) Antes de la rotación

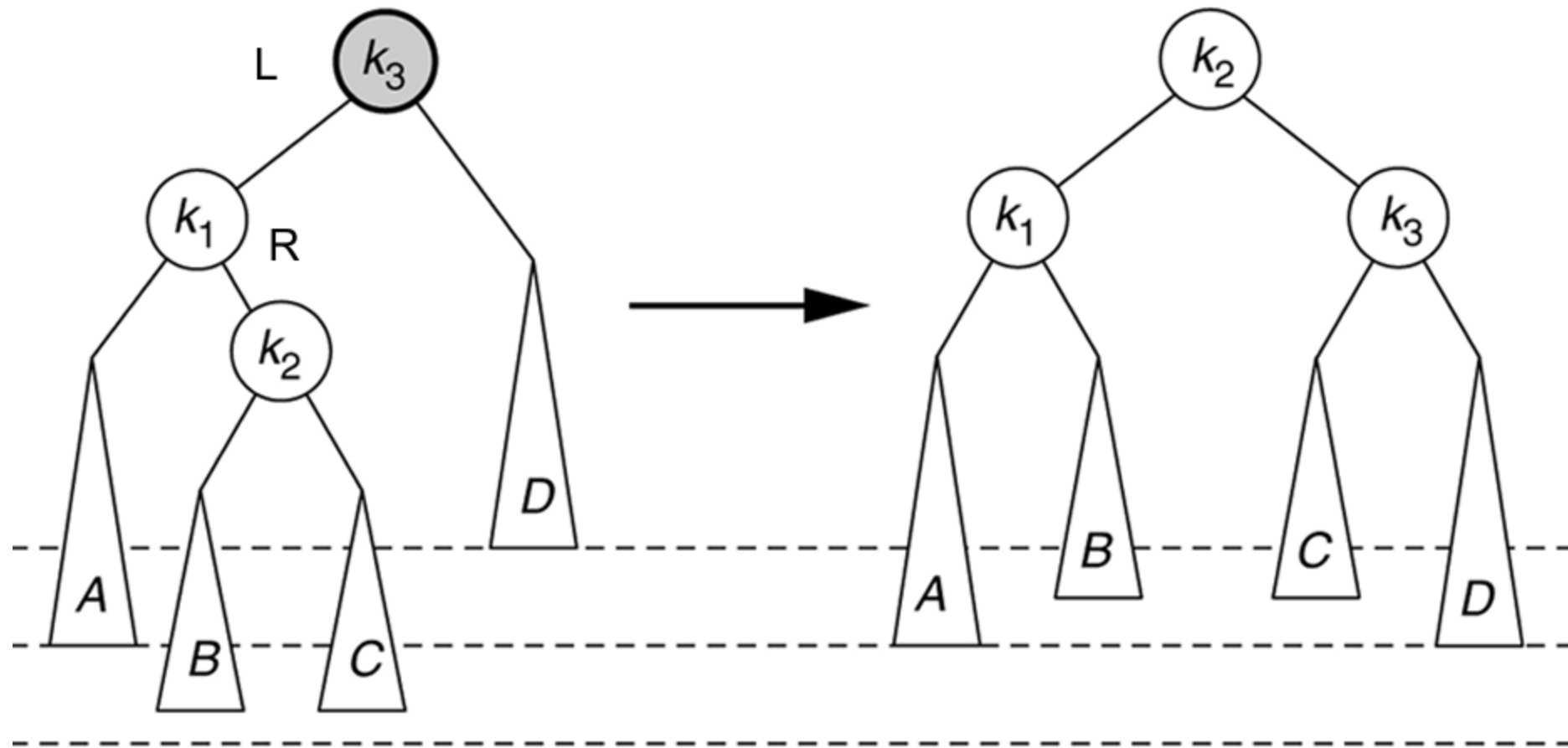
La rotación simple no resuelve el caso 2



a) Antes de la rotación

b) Después de la rotación

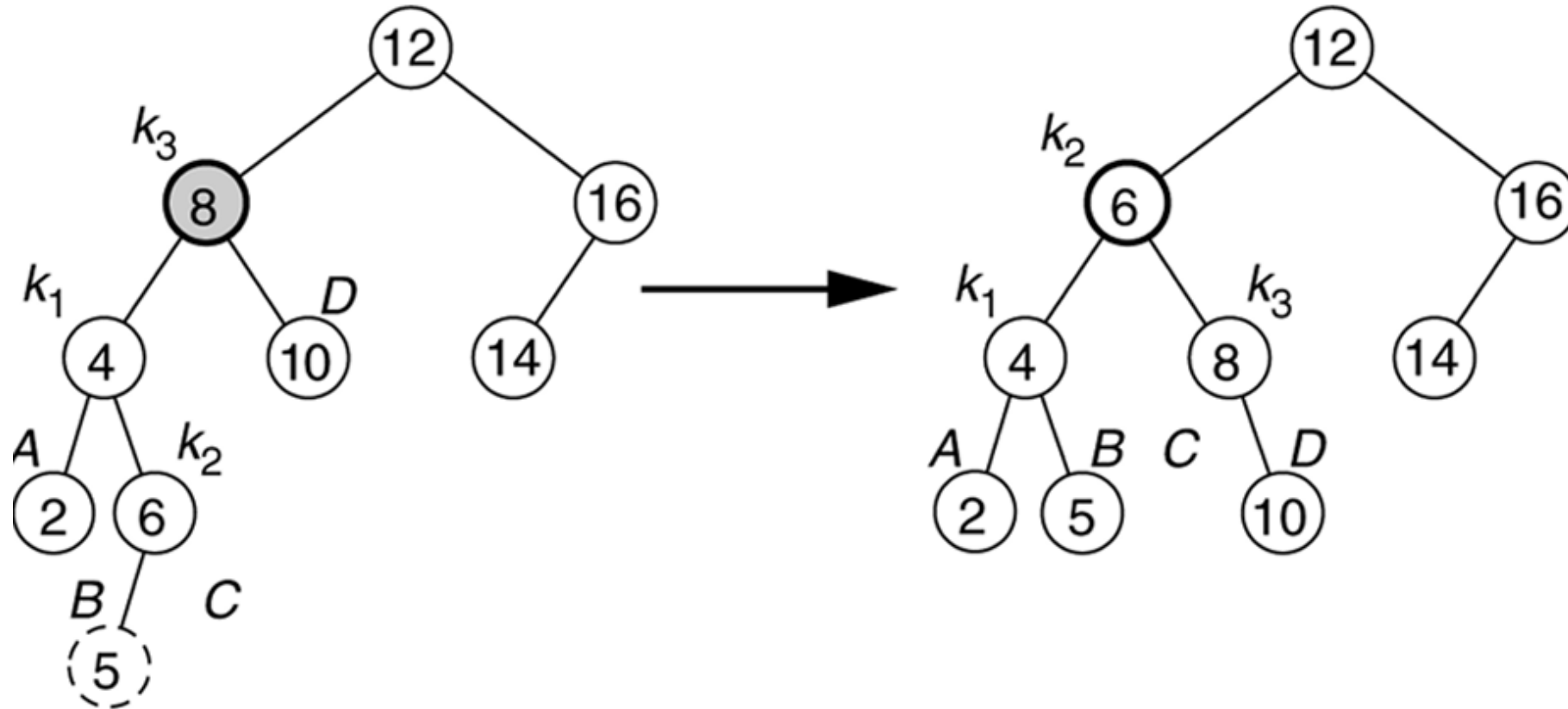
La rotación doble LR para resolver el caso 2



a) Antes de la rotación

b) Después de la rotación

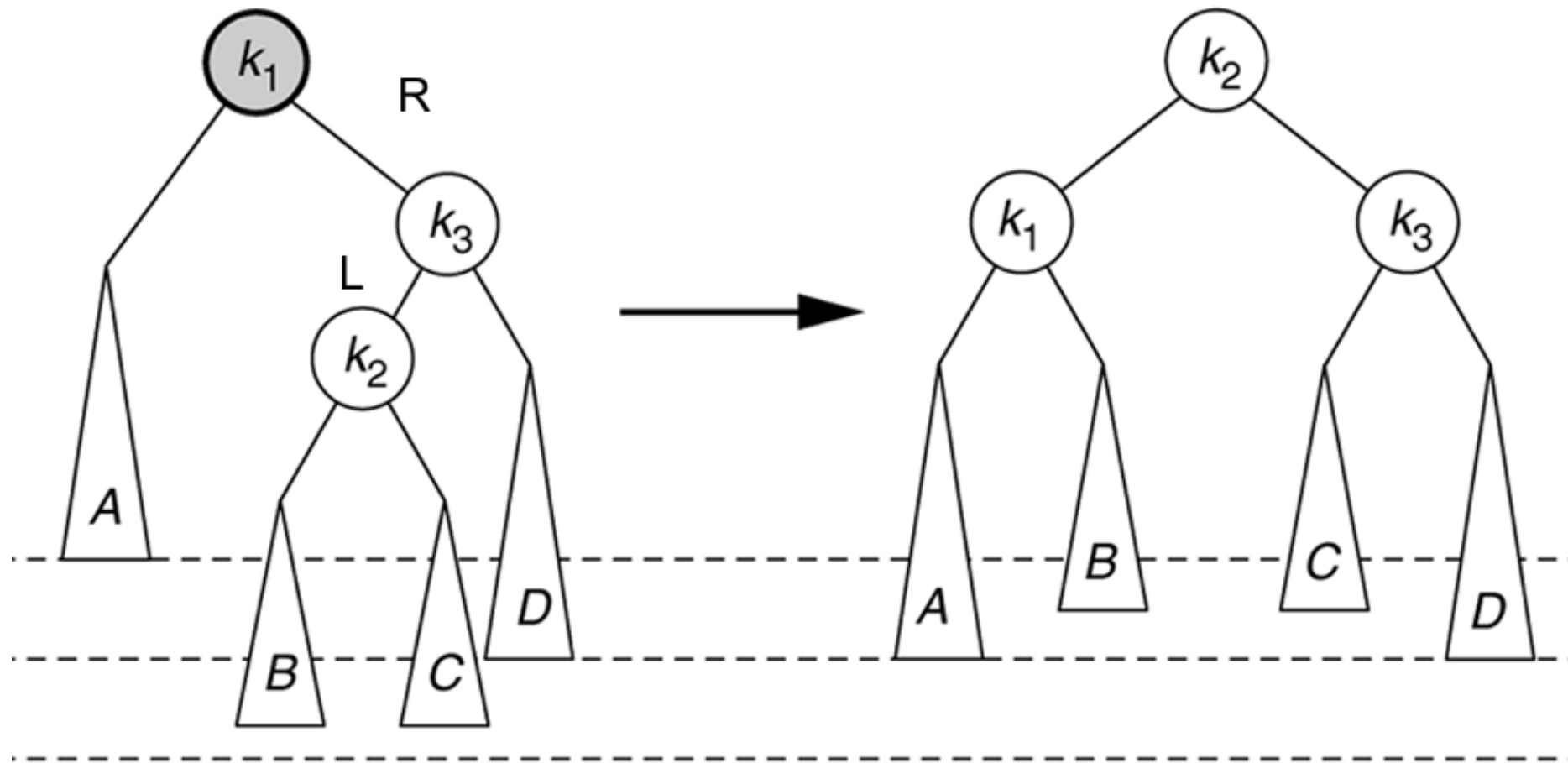
Rotación doble restablece el balance después de la inserción del 5.



a) Antes de la rotación

b) Después de la rotación

La rotación doble RL para resolver el caso 3



a) Antes de la rotación

b) Después de la rotación

Implementación de rotaciones simples en TElementoAB<T>

```
// CASO 1 LL
public TElementoAB<T> rotacionLL(TElementoAB<T> k2){
    TElementoAB<T> k1 = k2.getHijoIzq();
    k2.setHijoIzq(k1.getHijoDer());
    k1.setHijoDer(k2);
    return k1;
}

// CASO 4 RR
public TElementoAB<T> rotationRR(TElementoAB<T> k1){
    TElementoAB<T> k2 = k1.getHijoDer();
    k1.setHijoDer(k2.getHijoIzq());
    k2.setHijoIzq(k1);
    return k2;
}
```

Implementación de rotaciones dobles en TElementoAB<T>

```
// CASO 2 LR
```

```
public TElementoAB<T> rotationLR(TElementoAB<T> k3){  
    k3.setHijoIzq(rotacionRR(k3.getHijoIzq()));  
    return rotacioLL(k3);  
}
```

```
// CASO 3 RL
```

```
public TElementoAB<T> rotationRL(TElementoAB<T> k1){  
    k1.setHijoDer(rotacionLL(k1.getHijoDer()));  
    return rotacionRR(k1);  
}
```

Eliminación en AVL

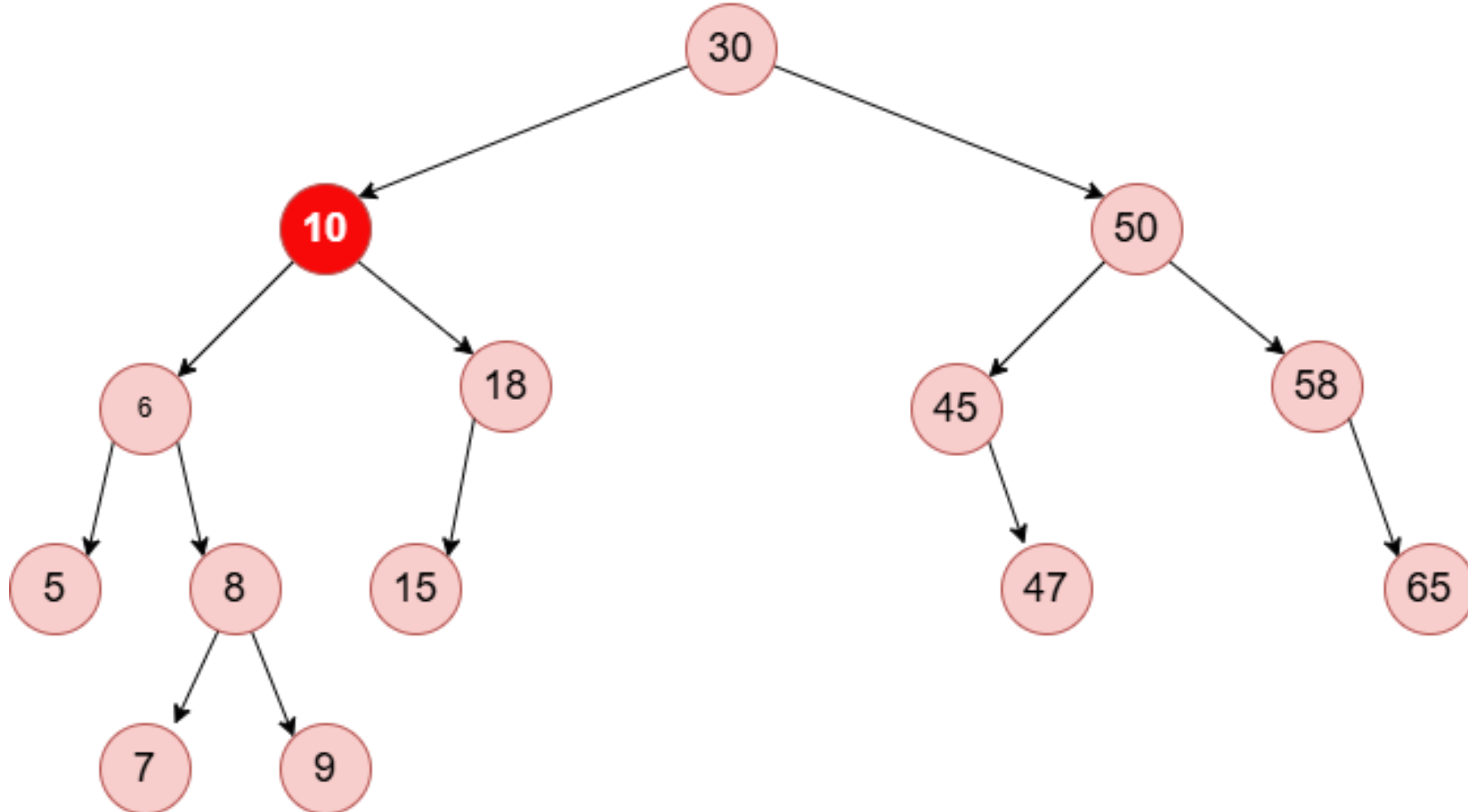
- La inserción como mucho rota una vez en el primer desbalance.
- La eliminación puede requerir aplicar varias rotaciones hasta llegar a la raíz del árbol.
- Se utiliza algoritmo de eliminación de AB.

Eliminación en AVL

- Ubicar el nodo a eliminar y reemplazarlo por el mayor nodo del subárbol izquierdo.
- Tras borrar se necesita re-balancear desde el nodo desbalanceado hasta la raíz:
 - Calcular $\text{balance} = \text{altura}(\text{izq}) - \text{altura}(\text{der})$
 - Si $\text{balance} > 1$ (muy cargado a la izquierda)
 - LL: si $\text{balance}(\text{izq}) \geq 0$
 - LR: si $\text{balance}(\text{izq}) < 0$
 - Si $\text{balance} < -1$ (muy cargado a la derecha)
 - RR: si $\text{balance}(\text{der}) \leq 0$
 - RL: si $\text{balance}(\text{der}) > 0$

Eliminación en AVL - Ejemplo

val_to_delete = 10



Ejercicios árboles AVL

- ¿Qué es un Arbol Binario Balanceado–AVL? Describa las situaciones de desbalanceo posibles(ilustre con ejemplos) y qué operaciones se deben aplicar en cada caso para restituir la condición de AVL.
- Defina árbol balanceado. Describa una estructura de datos apta para implementarlo. Desarrolle el algoritmo de búsqueda e inserción en árbol balanceado. Ilustre el funcionamiento mediante un ejemplo. ¿Cuáles el orden del tiempo de ejecución del mismo?
- Insertar en un árbol AVL las siguientes claves, en el orden indicado,
 - 8, 5, 2, 1, 20, 15, 16, 17,7 mostrando la evolución del árbol. Luego eliminarlas clave 15 y 7, mostrando los pasos.

Ejercicios árboles AVL

- Muestre el resultado de insertar en un árbol AVL las siguientes claves, en el orden indicado,
 - 2,1,4,5,9,3,6, 7 mostrando la evolución del árbol. Luego muestre el resultado de eliminar las clave 4, 5 y 9, mostrando la evolución
- Insertar en un árbol AVL las siguientes claves, en el orden indicado
 - 1,88,77,4,5,9,18,56,33,21,45,99,22 mostrando la evolución del árbol. Luego eliminarlas clave 18, 77, 1 y 45 mostrando los pasos.